

COMP1007- Programming Design and Implementation

Name: Isini Ayansa Perera

CurtinID: 23601321

University: Curtin University Colombo

CLASS: Details

FIELDS

courseEnrolled: String

yearLevel: Integer

cwa: Double

status: String

creditsEarned: Integer

DEFAULT CONSTRUCTOR

IMPORT: none

EXPORT: none

ASSERTION: Gives the object starting values

ALGORITHM:

BEGIN

SET courseEnrolled <- ""

SET yearLevel <- 1

SET cwa <- 0.0

SET status <- "FT"

SET creditsEarned <- 0

END

Details- CONSTRUCTOR

IMPORT: courseIn: STRING, yearIn: INTEGER, cwaIn: REAL, statusIn: STRING, creditsIn: INTEGER

EXPORT: none

ASSERTION: Builds a details record using setters to apply validation

ALGORITHM

```
BEGIN

CALL setCourseEnrolled(courseIn)

CALL setYearLevel(yearIn)

CALL setCwa(cwaIn)

CALL setStatus(statusIn)

CALL setCreditsEarned(creditsIn)

END
```

METHOD: setCourseEnrolled

```
IMPORT: courseEnrolledInput: STRING

EXPORT: none

ASSERTION: Course cannot be empty

ALGORITHM
```

```
BEGIN

IF courseEnrolledInput = NULL OR TRIM(courseEnrolledInput) = "" THEN

    RAISE ERROR "Course cannot be empty"

ENDIF

courseEnrolled <- TRIM(courseEnrolledInput)

END
```

METHOD: setYearLevel

```
IMPORT: yearLevelInput: INTEGER

EXPORT: none

ASSERTION: Year level must be between 1 and 4

ALGORITHM

BEGIN

IF yearLevelInput < 1 OR yearLevelInput > 4 THEN
```

```
    RAISE ERROR "Year Level must be between 1 and 4"
ENDIF
yearLevel <- yearLevelInput
END
```

METHOD: setCwa

```
IMPORT: cwaInput: REAL
EXPORT: none
ASSERTION: CWA must be between 0 and 100.
ALGORITHM

BEGIN
    IF cwaInput < 0 OR cwaInput > 100 THEN
        RAISE ERROR "CWA must be between 0 and 100"
    ENDIF
    cwa <- cwaInput
END
```

METHOD: setStatus

```
IMPORT: statusInput: STRING
EXPORT: none
ASSERTION: Status must be "FT" or "PT" (case insensitive input allowed)
ALGORITHM

BEGIN
    IF statusInput = NULL THEN
        RAISE ERROR "Status cannot be empty"
    ENDIF
    statusInput <- UPPER(TRIM(statusInput))
```

```
IF statusInput ≠ "FT" AND statusInput ≠ "PT" THEN
    RAISE ERROR "Status must be FT or PT"
ENDIF
Status <- statusInput
END
```

METHOD: setCreditsEarned

```
IMPORT: creditsEarnedInput: INTEGER
EXPORT: none
ASSERTION: Credits must be between 0 and 400.
ALGORITHM

BEGIN
    IF creditsEarnedInput < 0 OR creditsEarnedInput > 400 THEN
        RAISE ERROR "Credits earned must be between 0 and 400"
    ENDIF
    creditsEarned <- creditsEarnedInput
END
```

METHOD: getCourseEnrolled / getYearLevel / getCwa / getStatus / getCreditsEarned

```
IMPORT: none
EXPORT: corresponding field
ASSERTION: Simple accessors.
ALGORITHM

BEGIN
    RETURN field
END
```

METHOD: isEligibleToGraduate

IMPORT: none

EXPORT: BOOLEAN

ASSERTION: Eligible at 400 credits or more.

ALGORITHM

BEGIN

 RETURN (creditsEarned $\geq$ 400)

END

METHOD: toCSV

IMPORT: none

EXPORT: STRING

ASSERTION: Produces "courseEnrolled,yearLevel,Cwa,status,creditsEarned".

ALGORITHM

BEGIN

 RETURN courseEnrolled + "," + TO_STRING(yearLevel) + "," + TO_STRING(Cwa) + "," + status
 + "," + TO_STRING(creditsEarned)

END

CLASS: Student

FIELDS

studentID: String

firstName: String

lastName: String

details: Details

DEFAULT CONSTRUCTOR

```
IMPORT: none
EXPORT: none
ASSERTION: Initializes a blank but valid student with default Details.
ALGORITHM

BEGIN
SET studentID <- ""
SET firstName <- ""
SET lastName <- ""
SET details <- NEW Details()
END
```

CONSTRUCTOR- studentID, firstName, lastName, details

```
IMPORT: studentIDIn: String, firstNameIn: String, lastNameIn: String, detailsIn: Details
EXPORT: none
ASSERTION: Builds a Student using validated values.
ALGORITHM
```

```
BEGIN
CALL setStudentID(studentID)
CALL setFirstName(firstName)
CALL setLastName(lastName)
CALL setDetails(details)
END
```

METHOD- setStudentID

```
IMPORT: studentIDIn: STRING
EXPORT: none
ASSERTION: ID must be non-null and not empty after trim.
ALGORITHM
```

```
IF studentIDIn = NULL OR TRIM(studentIDIn)== "" THEN
    RAISE ERROR "StudentID cannot be empty"
ENDIF

studentID <- TRIM(studentIDIn)

END
```

METHOD- setFirstName

IMPORT: firstName: STRING

EXPORT: none

ASSERTION: First name must be non-null and not empty after trim.

ALGORITHM

IF firstNameIn = NULL OR TRIM(firstNameIn) = "" THEN

 THROW ERROR "First name cannot be empty"

ENDIF

 firstName <- TRIM(firstNameIn)

END

METHOD- setLastName

IMPORT: lastNameIn: STRING

EXPORT: none

ASSERTION: Last name must be non-null and not empty after trim.

ALGORITHM

IF lastNameIn = NULL OR TRIM(lastNameIn) = "" THEN

 RAISE ERROR "Last name cannot be empty"

ENDIF

 lastName = TRIM(lastNameIn)

END IF

METHOD: setDetails

IMPORT: detailsIn: Details

EXPORT: none

ASSERTION: Details object must not be null.

ALGORITHM

```
BEGIN  
  
IF detailsIn = NUL THEN  
    RAISE error "Details cannot be empty"  
ENDIF  
  
    details = detailsI  
  
END
```

METHOD: getStudentID

```
IMPORT: none  
EXPORT: String  
ASSERTION: Returns stored student ID.  
ALGORITHM  
  
RETURN studentID
```

METHOD: getFirstName

```
IMPORT: none  
EXPORT: String  
ASSERTION: Returns stored first name.  
ALGORITHM  
  
RETURN firstName
```

METHOD: getLastName

```
IMPORT: none  
EXPORT: String  
ASSERTION: Returns stored last name.  
ALGORITHM  
  
RETURN lastName
```

METHOD: getDetails

```
IMPORT: none  
EXPORT: Details  
ASSERTION: Returns the Details object.  
ALGORITHM
```

RETURN details

METHOD: toCSV

IMPORT: none

EXPORT: STRING

ASSERTION: Produces one CSV line for this student.

ALGORITHM:

BEGIN

 RETURN studentID + "," + firstName + "," + lastName + "," + details.toCSV()

END

CLASS: StudentSystem

FIELDS

maxNum: Integer < 1000

students: Array of Student [maxNum]

count : Integer <- 0

MAIN METHOD

IMPORT: none

EXPORT: none

ASSERTION: Runs the program and shows the main menu loop.

ALGORITHM:

BEGIN

CALL readStudentsFromFile("data.csv")

running <- TRUE

WHILE running DO

 CALL DISPLAYMenu()

```
choice <- readInt("Enter option: ")
```

```
SELECT choice
```

```
  CASE 1:
```

```
    CALL addStudent()
```

```
    CALL writeStudentsToFile("data.csv")
```

```
  CASE 2:
```

```
    CALL editStudent()
```

```
    CALL writeStudentsToFile("data.csv")
```

```
  CASE 3:
```

```
    CALL viewAllStudents()
```

```
  CASE 4:
```

```
    CALL filterByCourse()
```

```
  CASE 5:
```

```
    CALL filterByStatus()
```

```
  CASE 6:
```

```
    CALL showHighestCWA()
```

```
  CASE 7:
```

```
    CALL showAverageCWAPerCourse()
```

```
  CASE 8:
```

```
    CALL showGraduationEligibility()
```

```
  CASE 9:
```

```
    running <- FALSE
```

```
    CALL writeStudentsToFile("data.csv")
```

```
  DEFAULT:
```

```
    DISPLAY "Invalid choice"
```

```
END SELECT
```

```
END WHILE
```

```
END
```

METHOD: DISPLAYMenu

IMPORT: none

EXPORT: none

ASSERTION: Displays menu options for the user.

ALGORITHM:

BEGIN

DISPLAY "=====

DISPLAY " Welcome to Student Central "

DISPLAY "=====

DISPLAY "Your options for this system are listed below"

DISPLAY "1> Add new student"

DISPLAY "2> Edit student"

DISPLAY "3> View all students"

DISPLAY "4> Filter by course"

DISPLAY "5> Filter by status"

DISPLAY "6> Highest CWA"

DISPLAY "7> Average CWA for each course"

DISPLAY "8> Credit Analysis"

DISPLAY "9> Exit"

END

METHOD: readStudentsFromFile

IMPORT: filename: STRING

EXPORT: none

ASSERTION: Reads student records from CSV file and stores them in array.

ALGORITHM:

BEGIN

count <- 0

TRY OPEN fileName FOR READ

```

READ first line (header)

WHILE next line EXISTS AND count < maxNum DO

    SPLIT line BY "," INTO parts[]

    IF LENGTH(parts) < 8 THEN

        DISPLAY "Skipping row"

        CONTINUE

    END IF

    TRY

        SET studentID    = TRIM(parts[0])

        SET firstName    = TRIM(parts[1])

        SET lastName     = TRIM(parts[2])

        SET courseEnrolled = TRIM(parts[3])

        SET yearLevel    = PARSE_INT(TRIM(parts[4]))

        SET cwa          = PARSE_DOUBLE(TRIM(parts[5]))

        SET status       = TRIM(parts[6])

        SET creditsEarned = PARSE_INT(TRIM(parts[7]))

        CREATE new Details object d

        CALL d.setCourseEnrolled(courseEnrolled)

        CALL d.setYearLevel(yearLevel)

        CALL d.setCwa(cwa)

        CALL d.setStatus(status)

        CALL d.setCreditsEarned(creditsEarned)

        CREATE new Student s

        SET s = NEW Student(studentID, firstName, lastName, d)

        SET students[count] = s

        SET count = count + 1

    CATCH ANY ERROR

```

```
        DISPLAY "Error: skipped: "
    END TRY
END WHILE
CATCH FILE NOT FOUND
    DISPLAY "CSV not found, file will be created later"
END TRY
```

METHOD: writeStudentsToFile

```
IMPORT: filename: STRING
EXPORT: none
ASSERTION: Saves all student data into CSV file.
ALGORITHM:
```

```
BEGIN
OPEN fileName FOR WRITE
DISPLAY
"StudentID,FirstName,LastName,CourseEnrolled,YearLevel,CWA,Status,CreditsEarned"
FOR i <- 0 TO count - 1 DO
    DISPLAY students[i].toCSV()
END FOR
END
```

METHOD: viewAllStudents

```
IMPORT: none
EXPORT: none
ASSERTION: Displays all students or message if empty.
ALGORITHM:
```

```
BEGIN
IF count = 0 THEN
    DISPLAY "No students available"
ELSE
```

```
FOR i <- 0 TO count - 1 DO
    DISPLAY students[i].toCSV()
END FOR
END
```

METHOD: filterByCourse

IMPORT: none
EXPORT: none
ASSERTION: DISPLAYs students from a chosen course.
ALGORITHM:

```
BEGIN
SET searchCourse <- LOWER(TRIM(readLine("Enter a course: ")))
SET matchFound <- FALSE
FOR i <- 0 TO count - 1 DO
    SET course = LOWER(students[i].getDetails().getCourseEnrolled())
    IF course = searchCourse THEN
        DISPLAY students[i].toCSV()
        SET matchFound = TRUE
    END IF
END FOR
IF NOT matchFOUND THEN
    DISPLAY "No students found for " + searchCourse
END IF
```

METHOD: filterByStatus

IMPORT: none
EXPORT: none
ASSERTION: Filters students of FT or PT status.
ALGORITHM:

```
SET st <- UPPER(TRIM(readLine("Enter status (FT/PT): ")))
```

```
IF st != "FT" AND st != "PT" THEN
```

```
    DISPLAY "Input Invalid"
```

```
    RETURN
```

```
END IF
```

```
SET matchFound <- FALSE
```

```
FOR i <- 0 TO count - 1 DO
```

```
    IF students[i].getDetails().getStatus() = st THEN
```

```
        DISPLAY students[i].toCSV()
```

```
        SET matchFound <- TRUE
```

```
    END IF
```

```
END FOR
```

```
IF NOT matchFound THEN
```

```
    DISPLAY "No students found with status: " + st
```

```
END IF
```

METHOD: addStudent

```
IMPORT: none
```

```
EXPORT: none
```

```
ASSERTION: Adds a new student after validation.
```

```
ALGORITHM:
```

```
IF count >= maxNum THEN
```

```
    DISPLAY "No space"
```

```
    RETURN
```

```
END IF
```

```
REPEAT
```

```
    SET id <- TRIM(readLine("Student ID: "))
```

```

IF id = "" THEN
    DISPLAY "StudentID cannot be empty"
    CONTINUE
END IF

IF indexOfId(id) ≠ -1 THEN
    DISPLAY "ID already exists"
    CONTINUE
END IF

BREAK

UNTIL TRUE

SET first <- readNonEmpty("First name: ")
SET last <- readNonEmpty("Last name: ")
SET d <- NEW Deails()

REPEAT
    TRY CALL d.setCourseEnrolled(readNonEmpty("Course Enrolled: "))
    CATCH DISPLAY error "Error: " +MESSAGE
UNTIL success

REPEAT
    TRY CALL d.setYearLevel(readInt("Year level (1–4): ", 1, 4))
    CATCH DISPLAY error "Error: " +MESSAGE
UNTIL success

REPEAT
    TRY CALL d.setCwa(readDouble("CWA (0–100): ", 0, 100))
    CATCH DISPLAY error "Error: " +MESSAGE
UNTIL success

```

REPEAT

 TRY CALL d.setStatus(readNonEmpty("Status (FT/PT): "))

 CATCH DISPLAY error "Error: " +MESSAGE

UNTIL success

REPEAT

 TRY CALL d.setCreditsEarned(readInt("Credits (0–400): ", 0, 400))

 CATCH DISPLAY error "Error: " +MESSAGE

UNTIL success

TRY

CREATE new Student s = NEW Student(id, first, last, d)

SET students[count] = s

SET count = count + 1

DISPLAY "Student added successfully"

CATCH ANY => DISPLAY "Failed to add Student: " +MESSAGE

END

METHOD: editStudent

IMPORT: none

EXPORT: none

ASSERTION: Edits existing student data, invalid input is ignored with a message

ALGORITHM

BEGIN

SET id = TRIM(readLine("Enter Student ID: "))

SET indexId <- indexOfId(id)

IF indexId = -1 THEN

 DISPLAY "Student not found"

 RETURN END IF

```
SET s <- students[idx]
SET d <- s.getDetails()
DISPLAY "Editing: " + s.toCSV()
```

```
SET newVal = readLine("New first name (" + s.getFirstName() + "): ")
IF TRIM(newVal) ≠ "" THEN CALL s.setFirstName(newVal)
CATCH DISPLAY "Ignored: " +MESSAGE
ENDIF
```

```
SET newVal = readLine("New last name (" + s.getLastName() + "): ")
IF TRIM(newVal) ≠ "" THEN CALL s.setLastName(newVal)
CATCH DISPLAY "Ignored: " +MESSAGE
ENDIF
```

```
SET newVal = readLine("New course (" + d.getCourseEnrolled() + "): ")
IF TRIM(newVal) ≠ "" THEN CALL d.setCourseEnrolled(newVal)
CATCH DISPLAY "Ignored: " +MESSAGE
ENDIF
```

```
SET newVal = readLine("New year (" + d.getYearLevel() + "): ")
IF TRIM(newVal) ≠ "" THEN CALL d.setYearLevel(PARSE_INT(TRIM(newVal)))
CATCH DISPLAY "Ignored: Year must be 1 to 4 "
ENDIF
```

```
SET newVal = readLine("New CWA (" + d.getCwa() + "): ")
IF TRIM(newVal) ≠ "" THEN CALL d.setCwa(PARSE_DOUBLE(TRIM(newVal)))
CATCH DISPLAY "Ignored: CWA must be 0 to 100 "
ENDIF
```

```
SET newVal = readLine("New status (FT/PT) (" + d.getStatus() + "): ")
```

```
IF TRIM(newVal) ≠ "" THEN CALL d.setStatus(newVal)
```

```
CATCH DISPLAY "Ignored: " +MESSAGE
```

```
ENDIF
```

```
SET newVal = readLine("New credits (" + d.getCreditsEarned() + "): ")
```

```
IF TRIM(newVal) ≠ "" THEN CALL d.setCreditsEarned(PARSE_INT(TRIM(newVal)))
```

```
CATCH DISPLAY "Ignored: Credits earned must be 0 to 400 "
```

```
ENDIF
```

```
DISPLAY "Edit complete"
```

```
END
```

METHOD: showHighestCWA

```
IMPORT: none
```

```
EXPORT: none
```

```
ASSERTION: Finds and DISPLAYs students with the highest CWA.
```

```
ALGORITHM
```

```
BEGIN
```

```
IF count = 0 THEN
```

```
    DISPLAY "No students "
```

```
    RETURN
```

```
END IF
```

```
SET max <- students[0].getDetails().getCwa()
```

```
FOR i <- 1 TO count - 1 DO
```

```
    IF students[i].getDetails().getCwa() > max THEN
```

```
        SET max <- students[i].getDetails().getCwa()
```

```

    END IF
END FOR

shown <- 0

FOR i < 0 TO count -1 DO
    IF students[i].getDetails().getCwa() = max THEN
DISPLAY students[i].toCSV()
    shown <- shown + 1
ENDIF
ENDFOR

DISPLAY "Highest CWA = " + FORMAT(max,2) + " (students shown: " + shown + ")"
END

```

METHOD: showAverageCWAPerCourse

```

IMPORT: none
EXPORT: none
ASSERTION: Calculates average CWA for each course.
ALGORITHM:

BEGIN

    IF count = 0 THEN DISPLAY "No students"; RETURN ENDIF

    courses <- new ARRAY OF STRING size maxNum; nCourses <- 0

    FOR i <- 0 TO count -1 DO

        c <- students[i].getDetails().getCourseEnrolled()

        IF c = NULL THEN

            c <- "" ENDIF

        seen <- FALSE

        FOR j 0 TO nCourses-1 DO

            IF EQUALS_IGNORE_CASE(courses[j], c) THEN seen <- TRUE; BREAK ENDIF

        ENDFOR

        IF NOT seen THEN courses[nCourses] <- c; nCourses <- nCourses + 1 ENDIF
    
```

```

ENDFOR
FOR j <- 0 TO nCourses-1 DO
    target <- courses[j]; sum <- 0.0; n <- 0
    FOR i <- 0 TO count-1 DO
        c <- students[i].getDetails().getCourseEnrolled()
        IF c ≠ NULL AND EQUALS_IGNORE_CASE(c, target) THEN
sum <- sum + students[i].getDetails().getCwa(); n <- n + 1 ENDIF
    ENDFOR
    avg <- (n = 0) ? 0.0 : sum / n
    DISPLAY target + " -> Average CWA: " + FORMAT(avg,2) + " (n=" + n + ")"
ENDFOR
END

```

METHOD: showGraduationEligibility

```

IMPORT: none
EXPORT: none
ASSERTION: Lists students with credits ≥ 400.
ALGORITHM:

```

```

BEGIN
    IF count = 0 THEN
        DISPLAY "No Students"
    RETURN ENDIF
    eligible <- 0
    FOR I <- 0 TO count -1 DO
        Ok <- students[i].getDetails().isEligibleToGraduate()
        IF ok THEN eligible <- eligible + 1 ENDIF
        DISPLAY "Student ID: " + students[i].getStudentID()
        DISPLAY "Student First Name: " + students[i].getFirstName()
        DISPLAY "Student Last Name: " + students[i].getLastName()
    ENDFOR
END

```

```

    DISPLAY "Eligibility: " + (ok ? "Eligible" : "Not Eligible")
ENDFOR

DISPLAY "Eligible to graduate: " + eligible + " out of " + count
END

```

METHOD: indexOfId

```

IMPORT: id: STRING
EXPORT: INTEGER
ASSERTION: Returns array index of studentID or -1
ALGORITHM

```

```

BEGIN

SET key <- TRIM(id)

FOR i <- 0 TO count - 1 DO

    IF students[i].getStudentID() = key THEN

        RETURN i

    END IF

END FOR

RETURN -1

END

```

METHOD: readLine / readInt / readNonEmpty / readDouble

```

IMPORT / EXPORT: per input type
ASSERTION: Reads and validates user input.
ALGORITHM (summary):

```

```

readLine(prompt): DISPLAY prompt; RETURN input text or "" on error
readInt(prompt): LOOP parse INT when fail DISPLAY warning RETRY
readNonEmpty(prompt): LOOP until TRIM(value) ≠ "" RETURN value
readInt(prompt, min, max): LOOP readInt, if out of range, warns and retry
readDouble(prompt, min, max): LOOP Parse REAL, if out of range, warns and retry

```